Contents

1 Algebra **1**
1.1 Gauss . . . 1

2 Combinatorics **1**
2.1 Gray Code . . . 1

3 Convex Hull Trick **2**
3.1 Convex Hull Trick . . . 2
3.2 Li Chao Tree . . . 2

4 Convolutions **3**
4.1 FFT MOD . . . 3
4.2 FFT . . . 3
4.3 FHT . . . 4
4.4 Multipoint Evaluation . . . 4
4.5 Poly Division Mod . . . 5
4.6 Poly Exp . . . 5
4.7 Poly Log . . . 5
4.8 SOS DP . . . 5

5 Data Structures **6**
5.1 Centroid Decomposition . . . 6
5.2 DSU . . . 6
5.3 Disjoint Sparse Table . . . 7
5.4 Fenwick 2D . . . 7
5.5 Fenwick Tree . . . 7
5.6 HLD . . . 8
5.7 Implicit Segment Tree . . . 8
5.8 Persistent DSU . . . 9
5.9 Persistent Segment Tree . . . 10
5.10 Segment Tree Beats . . . 10
5.11 Treap . . . 12

6 Flows **13**
6.1 Dinic . . . 13
6.2 Min Cost Max Flow Capacity Scaling . . . 13
6.3 Min Cost Max Flow . . . 14

7 Geometry **15**
7.1 Basic . . . 15
7.2 Halfplanes Intersection . . . 16

8 Graphs **17**
8.1 2 SAT . . . 17
8.2 Euler Path . . . 17
8.3 Hopcroft Karp . . . 17
8.4 Hungarian . . . 18
8.5 Initial Pairing . . . 18
8.6 Kuhn . . . 19

9 Matroids **19**
9.1 MatroidExamples . . . 19
9.2 MatroidIntersectionUnweighted . . . 22
9.3 MatroidIntersectionWeighted . . . 22

10 Number Theory **23**
10.1 CRT . . . 23
10.2 Eratosthenes . . . 23
10.3 FloorSum . . . 23
10.4 Miller Rabin . . . 23

11 Strings **24**
11.1 LCP . . . 24
11.2 Prefix Automate . . . 24
11.3 Prefix Function . . . 24
11.4 Suffix Massive . . . 25
11.5 Suffix Tree . . . 25
11.6 Z Function . . . 26

1 Algebra

1.1 Gauss

```
// a: n x (m + 1), last column is free terms
// returns one solution (free vars = 0), empty
    vector if no solutions
vector<ld> gauss(vector<vector<ld>> a) {
    const ld EPS = 1e-12;
    ll n = a.size();
    if (n == 0) {
        return {};
    }
    ll m = a[0].size() - 1;

    vector<ll> where(m, -1);
    for (ll col = 0, row = 0; col < m && row < n; ++
    col) {
        ll sel = row;
        for (ll i = row; i < n; ++i) {
            if (fabsl(a[i][col]) > fabsl(a[sel][col
    ]))
                sel = i;
        }
        if (fabsl(a[sel][col]) < EPS)
            continue;

        swap(a[sel], a[row]);
        where[col] = row;

        for (ll i = 0; i < n; ++i) {
            if (i == row)
                continue;
            ld c = a[i][col] / a[row][col];
            if (fabsl(c) < EPS)
                continue;
            for (ll j = col; j <= m; ++j)
                a[i][j] -= a[row][j] * c;
        }
        ++row;
    }

    vector<ld> ans(m, 0);
    for (ll i = 0; i < m; ++i) {
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i
    ];
    }

    for (ll i = 0; i < n; ++i) {
        ld sum = 0;
        for (ll j = 0; j < m; ++j)
            sum += ans[j] * a[i][j];
        if (fabsl(sum - a[i][m]) > EPS)
            return {};
    }
    return ans;
}
```

2 Combinatorics

2.1 Gray Code

```
inline ll g(ll n) {
    return n ^ (n >> 1);
}
```

3 Convex Hull Trick

3.1 Convex Hull Trick

```
// monotone lines
struct ConvexHullTrick {
    // a, b of length k, p of lengths max(0, k - 1)
    // all lines is a[i] * x + b[i]
    // and line i > 0 maximum from x = p[i - 1] to x
     = p[i] - 1
    vector<ll> a, b, p;

    // a1 < a2
    static ll intersect(ll a1, ll b1, ll a2, ll b2)
    {
        ll den = a2 - a1;
        ll num = b1 - b2;
        if (num >= 0) {
            return (num + den - 1) / den;
        } else {
            return num / den;
        }
    }

    // na >= a[i] forall i
    void add_line(ll na, ll nb) {
        if (a.empty()) {
            a.push_back(na);
            b.push_back(nb);
            return;
        }
        if (a.back() > na) {
            return;
        }
        while (!p.empty() && a.back() * p.back() + b
    .back() <= na * p.back() + nb) {
            p.pop_back();
            a.pop_back();
            b.pop_back();
        }
        // else na == a.back() and in one concrete
    point
        // new line leq of old > in all points new
    line >
        if (a.back() < na) {
            p.push_back(intersect(a.back(), b.back()
    , na, nb));
            a.push_back(na);
            b.push_back(nb);
        }
    }

    ll query(ll x) {
        if (a.empty()) {
            return -inf;
        }
        ll l = 0, r = a.size();
        while (r - l > 1) {
            ll m = (r + l) / 2;
            if (p[m - 1] <= x) {
                l = m;
            } else {
                r = m;
            }
        }
        return a[l] * x + b[l];
    }
};
```

3.2 Li Chao Tree

```
struct LiChaoMax {
    struct Line {
        ld a = 0;
        ld b = -inf;

        Line(ld A = 0, ld B = -inf) : a(A), b(B) {
        }

        ld get(ld x) {
            return a * x + b;
        }
    };

    ll n;
    vector<ll> c;
    vector<Line> tree;

    LiChaoMax(const vector<ll>& C) {
        c = C;
        n = c.size();
        tree.resize(4 * n + 3);
    }

    void add_line(ll v, ll l, ll r, Line line) {
        ll m = (l + r) / 2;

        bool lef = line.get(c[l]) > tree[v].get(c[l
    ]);
        bool mid = line.get(c[m]) > tree[v].get(c[m
    ]);

        if (mid) {
            swap(line, tree[v]);
        }

        if (l + 1 == r) {
            return;
        }

        if (lef != mid) {
            add_line(v * 2 + 1, l, m, line);
        } else {
            add_line(v * 2 + 2, m, r, line);
        }
    }

    void add_line(ld a, ld b) {
        add_line(0, 0, n, Line(a, b));
    }

    ld query(ll v, ll l, ll r, ll k) {
        ld res = tree[v].get(c[k]);

        if (l + 1 == r) {
            return res;
        }

        ll m = (l + r) / 2;

        if (k < m) {
            res = max(res, query(v * 2 + 1, l, m, k)
    );
        } else {
            res = max(res, query(v * 2 + 2, m, r, k)
    );
        }

        return res;
    }

    ld query_id(ll k) {
        return query(0, 0, n, k);
```

```
    }

    ld query_x(ll x) {
        ll k = lower_bound(all(c), x) - c.begin();
        return query_id(k);
    }
};
```

4 Convolutions

4.1 FFT MOD

```
const ll MOD = 998244353;
const ll ROOT = 31;
const int ROOT_PW = (1ll << 23);

vector<vector<ll>> w;

void precalc_ws(ll lg_n) {
    w.resize(lg_n + 1);
    w[0].resize(1, 1);
    for (int l = 1; l <= lg_n; ++l) {
        ll n = (1ll << l);
        w[l].resize(n / 2);

        ll fw = ROOT;
        for (int i = 0; i < 23 - l; ++i) {
            fw = (fw * fw) % MOD;
        }
        for (int j = 0; j < (n >> 1); ++j) {
            if (j % 2 == 0) {
                w[l][j] = w[l - 1][j / 2];
            } else {
                w[l][j] = (w[l - 1][j / 2] * fw) %
    MOD;
            }
        }
    }
}

void fft_inpl(vector<ll>& a, bool invert) {
    int n = a.size();

    // apply permutation by bit_rev
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2, lg = 1; len <= n; len <<= 1,
    ++lg) {
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < (len >> 1); ++j) {
                ll root = w[lg][j];
                if (invert && j) {
                    root = MOD - w[lg][(len >> 1) -
    j];
                }
                ll u = a[i + j], v = (a[i + j + len
    / 2] * root) % MOD;
                a[i + j] = u + v < MOD ? u + v : u +
     v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u
    - v : u - v + MOD;
            }
        }
    }
    if (invert) {
        int nrev = rev(n);
        for (int i = 0; i < n; ++i) {
            a[i] *= nrev;
            a[i] %= MOD;
        }
    }
}

vector<ll> vect_mult(const vector<ll>& a, const
    vector<ll>& b) {
    ll sz = a.size() + b.size() - 1;
    ll n = 1;
    int lg_n = 0;
    while (n < sz) {
        n <<= 1;
        ++lg_n;
    }
    if (w.size() <= lg_n) {
        precalc_ws(lg_n);
    }

    vector<ll> fa(n, 0), fb(n, 0);
    for (int i = 0; i < n; ++i) {
        if (i < a.size()) {
            fa[i] = a[i];
        }
        if (i < b.size()) {
            fb[i] = b[i];
        }
    }

    fft_inpl(fa, false);
    fft_inpl(fb, false);
    for (int i = 0; i < n; ++i) {
        fa[i] *= fb[i];
        fa[i] %= MOD;
    }
    fft_inpl(fa, true);

    vector<ll> mult(sz);
    for (int i = 0; i < sz; ++i) {
        mult[i] = fa[i];
    }
    return mult;
}
```

4.2 FFT

```
vector<vector<comp>> w;

void precalc_ws(ll lg_n) {
    w.resize(lg_n + 1);
    w[0].resize(1, 1);
    for (int l = 1; l <= lg_n; ++l) {
        ll n = (1ll << l);
        w[l].resize(n / 2);

        // need cosl, sinl for complex<ld>
        comp fw(cos(2 * PI / n), sin(2 * PI / n));
        for (int j = 0; j < (n >> 1); ++j) {
            if (j % 2 == 0) {
                w[l][j] = w[l - 1][j / 2];
            } else {
                w[l][j] = w[l - 1][j / 2] * fw;
            }
        }
    }
}

void fft_inpl(vector<comp>& a, bool invert) {
    int n = a.size();
```

```
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2, lg = 1; len <= n; len <<= 1,
    ++lg) {
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < (len >> 1); ++j) {
                comp root = w[lg][j];
                if (invert && j)
                    root = -w[lg][(len >> 1) - j];
                comp u = a[i + j], v = a[i + j + (
    len >> 1)] * root;
                a[i + j] = u + v;
                a[i + j + (len >> 1)] = u - v;
            }
        }
    }
    if (invert) {
        double rev_n = 1.0 / n;
        for (int i = 0; i < n; ++i) {
            a[i] *= rev_n;
        }
    }
}

vector<ll> vect_mult(const vector<ll>& a, const
    vector<ll>& b) {
    ll sz = a.size() + b.size() - 1;
    ll n = 1;
    int lg_n = 0;
    while (n < sz) {
        n <<= 1;
        ++lg_n;
    }
    if (w.size() <= lg_n) {
        precalc_ws(lg_n);
    }

    vector<comp> p(n, 0);
    for (int i = 0; i < n; ++i) {
        p[i] = comp(i < a.size() ? a[i] : 0, i < b.
    size() ? b[i] : 0);
    }

    fft_inpl(p, false);
    for (int i = 0; i < n; ++i) {
        p[i] *= p[i];
    }
    fft_inpl(p, true);

    vector<ll> mult(sz);
    for (int i = 0; i < sz; ++i) {
        // need roundl for complex<ld>
        mult[i] = round(p[i].imag() * 0.5);
    }
    return mult;
}
```

4.3 FHT

```
inline ll soft_mod(ll a) {
    return a >= MOD ? a - MOD : a;
}

void FHT(vector<ll>& a) {
    ll sz = a.size();
    for (int mid = 1; mid < sz; mid <<= 1) {
        for (int i = 0; i < sz; i += mid << 1) {
            for (int j = 0; j < mid; ++j) {
                ll x = a[i + j];
                ll y = a[i + j + mid];
                a[i + j] = soft_mod(x + y);
                a[i + j + mid] = soft_mod(x - y +
    MOD);
            }
        }
    }
}

vector<ll> xor_mult(vector<ll> a, vector<ll> b) {
    FHT(a);
    FHT(b);
    ll n = a.size();
    vector<ll> c(n);

    for (int i = 0; i < n; ++i) {
        c[i] = (a[i] * b[i]) % MOD;
    }

    FHT(c);
    ll revn = bin_pow(n, MOD - 2);
    for (int i = 0; i < n; ++i) {
        c[i] *= revn;
        c[i] %= MOD;
    }
    return c;
}
```

4.4 Multipoint Evaluation

```
int poly_deg(const vector<ll>& a) {
    int d = (int)a.size() - 1;
    while (d >= 0 && a[d] == 0)
        --d;
    return d;
}

vector<ll> poly_trim(vector<ll> a) {
    while (!a.empty() && a.back() == 0)
        a.pop_back();
    return a;
}

vector<ll> poly_rev(vector<ll> a, ll n) {
    a.resize(n, 0);
    reverse(a.begin(), a.end());
    return a;
}

// a % b, requires b.back() != 0
vector<ll> poly_mod(vector<ll> a, vector<ll> b) {
    a = poly_trim(a);
    b = poly_trim(b);
    int n = poly_deg(a), m = poly_deg(b);
    if (n < m)
        return a;
    ll k = n - m + 1;
    vector<ll> q = vect_mult(poly_rev(a, n + 1),
    inv_mod_xm(poly_rev(b, m + 1), k));
    q.resize(k, 0);
    reverse(q.begin(), q.end());
    vector<ll> qb = vect_mult(q, b);
    vector<ll> r(m, 0);
    for (int i = 0; i < m; ++i) {
        ll ai = i < a.size() ? a[i] : 0;
        ll bi = i < qb.size() ? qb[i] : 0;
        r[i] = (ai + MOD - bi % MOD) % MOD;
    }
```

```
    return poly_trim(r);
}

vector<vector<ll>> multipoint_tree;
vector<ll> multipoint_xs, multipoint_ys;

void multipoint_build(int v, int l, int r) {
    if (r - l == 1) {
        multipoint_tree[v] = {(MOD - multipoint_xs[l
    ] % MOD) % MOD, 1};
        return;
    }
    int mid = (l + r) / 2;
    multipoint_build(2 * v, l, mid);
    multipoint_build(2 * v + 1, mid, r);
    multipoint_tree[v] = vect_mult(multipoint_tree[2
     * v], multipoint_tree[2 * v + 1]);
}

void multipoint_dfs(int v, int l, int r, vector<ll>
    p) {
    p = poly_mod(p, multipoint_tree[v]);
    if (r - l == 1) {
        multipoint_ys[l] = p.empty() ? 0 : p[0];
        return;
    }
    int mid = (l + r) / 2;
    multipoint_dfs(2 * v, l, mid, p);
    multipoint_dfs(2 * v + 1, mid, r, p);
}

// evaluate poly p at xs[i]; |p| = n, |xs| = m, O((n
     + m) log^2 m)
vector<ll> multipoint_eval(vector<ll> p, vector<ll>
    xs) {
    int m = xs.size();
    if (m == 0)
        return {};
    multipoint_xs = xs;
    multipoint_ys.assign(m, 0);
    multipoint_tree.assign(4 * m, {});
    multipoint_build(1, 0, m);
    multipoint_dfs(1, 0, m, p);
    return multipoint_ys;
}
```

4.5 Poly Division Mod

```
vector<ll> cut_prefix(const vector<ll>& a, ll m) {
    vector<ll> r(m, 0);
    for (int i = 0; i < a.size() && i < m; ++i)
        r[i] = a[i];
    return r;
}

// b[0] invertible; returns b^{-1} mod x^m
vector<ll> inv_mod_xm(const vector<ll>& b, ll m) {
    if (m <= 0 || b.empty() || b[0] % MOD == 0)
        return {};
    vector<ll> inv(1, bin_pow((b[0] % MOD + MOD) %
    MOD, MOD - 2));
    ll len = 1;
    while (len < m) {
        ll nlen = min(2 * len, m);
        vector<ll> t = vect_mult(cut_prefix(b, nlen)
    , inv);
        t.resize(nlen, 0);
        for (int i = 0; i < nlen; ++i)
            t[i] = (MOD - t[i]) % MOD;
        t[0] = (t[0] + 2) % MOD;
        inv = vect_mult(inv, t);
        inv.resize(nlen, 0);
        len = nlen;
    }
    return inv;
}

// returns c, where c * b == a (mod x^m), requires b
    [0] != 0
vector<ll> div_mod_xm(const vector<ll>& a, const
    vector<ll>& b, ll m) {
    if (m <= 0)
        return {};
    return cut_prefix(vect_mult(cut_prefix(a, m),
    inv_mod_xm(b, m)), m);
}
```

4.6 Poly Exp

```
// a[0] == 0; returns exp(a) mod x^n
vector<ll> exp_mod_xm(const vector<ll>& a, ll n) {
    if (n <= 0)
        return {};
    vector<ll> f = {1};
    ll len = 1;
    while (len < n) {
        ll nlen = min(2 * len, n);
        vector<ll> g = cut_prefix(a, nlen);
        vector<ll> lf = log_mod_xm(f, nlen);
        for (int i = 0; i < nlen; ++i)
            g[i] = (g[i] + MOD - lf[i]) % MOD;
        g[0] = (g[0] + 1) % MOD;
        f = cut_prefix(vect_mult(f, g), nlen);
        len = nlen;
    }
    return f;
}
```

4.7 Poly Log

```
vector<ll> poly_deriv(const vector<ll>& a) {
    if (a.size() <= 1)
        return {0};
    vector<ll> r(a.size() - 1);
    for (int i = 1; i < a.size(); ++i)
        r[i - 1] = a[i] * i % MOD;
    return r;
}

vector<ll> poly_integr(const vector<ll>& a) {
    vector<ll> r(a.size() + 1);
    r[0] = 0;
    for (int i = 0; i < a.size(); ++i)
        r[i + 1] = a[i] * bin_pow(i + 1, MOD - 2) %
    MOD;
    return r;
}

// a[0] == 1; returns log(a) mod x^n
vector<ll> log_mod_xm(const vector<ll>& a, ll n) {
    if (n <= 0)
        return {};
    vector<ll> t = vect_mult(poly_deriv(cut_prefix(a
    , n)), inv_mod_xm(a, n));
    return cut_prefix(poly_integr(cut_prefix(t, n -
    1)), n);
}
```

4.8 SOS DP

```
void sos(vector<ll>& dp) {
    ll n = dp.size();
    ll k = 0;
    while ((1ll << k) < n) {
```

```
        ++k;
    }

    for (int j = 0; j < k; ++j) {
        for (int i = 0; i < n; ++i) {
            if (i & (1ll << j)) {
                dp[i] += dp[i ^ (1ll << j)];
            }
        }
    }
}

void revsos(vector<ll>& dp) {
    ll n = dp.size();

    ll k = 0;
    while ((1ll << k) < n) {
        ++k;
    }

    for (int j = k - 1; j >= 0; --j) {
        for (int i = 0; i < n; ++i) {
            if (i & (1ll << j)) {
                dp[i] -= dp[i ^ (1ll << j)];
            }
        }
    }
}
```

5 Data Structures

5.1 Centroid Decomposition

```
struct CentroidDecomp {
    ll n;
    vector<ll> parent;

    void calc_sizes(ll v, ll mark, vector<ll>& used,
     vector<vector<ll>>& g, vector<ll>& s) {
        s[v] = 1;
        used[v] = mark;

        for (ll u : g[v]) {
            if (used[u] < mark) {
                calc_sizes(u, mark, used, g, s);
                s[v] += s[u];
            }
        }
    }

ll find_centroid(ll v, ll mark, vector<ll>& used
, vector<vector<ll>>& g, vector<ll>& s, ll sz) {
    used[v] = mark;

    for (ll u : g[v]) {
        if (used[u] < mark) {
            if (s[u] > sz / 2) {
                return find_centroid(u, mark,
used, g, s, sz);
            }
        }
    }

    return v;
}

void set_parent(ll v, ll mark, vector<ll>& used,
 vector<vector<ll>>& g, ll cent) {
    if (v != cent)
        parent[v] = cent;
    used[v] = mark;

    for (ll u : g[v]) {
        if (used[u] < mark) {
            set_parent(u, mark, used, g, cent);
        }
    }
}

CentroidDecomp(ll N, vector<vector<ll>>& g) {
    n = N;
    parent.assign(n, -1);

    vector<ll> used(n, 0);
    vector<ll> s(n, 0);
    ll cnt_used = 0;
    ll mark = 0;

    while (cnt_used < n) {
        ++mark;
        ll pass_mark = mark;
        for (int i = 0; i < n; ++i) {
            if (used[i] >= pass_mark)
                continue;
            calc_sizes(i, pass_mark, used, g, s)
;
            ++mark;
            ll cent = find_centroid(i, mark,
 used, g, s, s[i]);

            ++mark;
            set_parent(i, mark, used, g, cent);
            used[cent] = inf;
            ++cnt_used;
        }
    }
};
```

5.2 DSU

```
struct DSU {
    ll n;
    vector<ll> prev, size;

    DSU(ll N) : n(N) {
        prev.assign(n, -1);
        size.assign(n, 1);
    }

    ll root(ll a) {
        ll aa = a;
        while (prev[a] != -1)
            a = prev[a];

        while (prev[aa] != -1) {
            ll last = prev[aa];
            prev[aa] = a;
            aa = last;
        }
        return a;
    }

    bool connected(ll a, ll b) {
        return root(a) == root(b);
    }

    bool unite(ll a, ll b) {
        a = root(a);
        b = root(b);
        if (a == b)
            return false;
        if (size[a] < size[b])
            swap(a, b);
```

```
        prev[b] = a;
        size[a] += size[b];
        return true;
    }
};
```

5.3 Disjoint Sparse Table

```
struct DisjointSparseTable {
    vector<vector<ll>> dp;

    ll neutral() {
        return 0;
    }
    ll merge(ll a, ll b) {
        return a + b;
    }

    DisjointSparseTable() {
    }

    DisjointSparseTable(const vector<ll>& a) {
        ll n = a.size();
        ll k = 0;
        ll nn = 1;
        while (nn <= n) {
            ++k;
            nn <<= 1;
        }

        dp.assign(k, vector<ll>(n + 1, neutral()));

        ll t = 0;
        for (ll len = 1; len <= n; len <<= 1, ++t) {
            for (ll cent = len; cent <= n; cent +=
    len * 2) {
                dp[t][cent - 1] = a[cent - 1];
                for (ll i = cent - 2; i >= cent -
    len; --i) {
                    dp[t][i] = merge(a[i], dp[t][i +
     1]);
                }

                dp[t][cent] = neutral();
                for (ll i = cent + 1; i < min(cent +
     len, n + 1); ++i) {
                    dp[t][i] = merge(dp[t][i - 1], a
    [i - 1]);
                }
            }
        }
    }

    // query on [l, r)
    ll query(ll l, ll r) {
        if (r <= l)
            return neutral();
        ll t = 63 - __builtin_clzll(l ^ r);
        return merge(dp[t][l], dp[t][r]);
    }
};
```

5.4 Fenwick 2D

```
template <class T>
struct Fenwick2D {
    ll n = 0, m = 0;
    vector<vector<T>> bit;

    Fenwick2D() = default;
    Fenwick2D(int n_, int m_) {
        init(n_, m_);
    }

    void init(int n_, int m_) {
        n = n_;
        m = m_;
        bit.assign(n + 1, vector<T>(m + 1, T{}));
    }

    // add delta at (x, y), 0-indexed
    void add(ll x, ll y, T delta) {
        for (ll i = x + 1; i <= n; i += i & -i)
            for (ll j = y + 1; j <= m; j += j & -j)
                bit[i][j] += delta;
    }

    // sum over [0..x] x [0..y], 0-indexed; returns
    0 if x<0 or y<0
    T sum_prefix(ll x, ll y) const {
        if (x < 0 || y < 0)
            return T{};
        T res{};
        for (ll i = x + 1; i > 0; i -= i & -i)
            for (ll j = y + 1; j > 0; j -= j & -j)
                res += bit[i][j];
        return res;
    }

    // sum over rectangle [x1..x2] x [y1..y2],
    inclusive, 0-indexed
    T sum_rect(ll x1, ll y1, ll x2, ll y2) const {
        return sum_prefix(x2, y2) - sum_prefix(x1 -
    1, y2) - sum_prefix(x2, y1 - 1) + sum_prefix(x1
    - 1, y1 - 1);
    }
};
```

5.5 Fenwick Tree

```
struct FenwickTree {
    vector<ll> tree;
    ll n;

    FenwickTree(ll N) {
        n = N;
        tree.assign(n, 0);
    }

    ll F(ll x) {
        return x & (x + 1);
    }

    ll H(ll x) {
        return x | (x + 1);
    }

    // [0, r)
    ll query(ll r) {
        if (r > n) {
            r = n;
        }
        --r;
        ll ans = 0;
        while (r >= 0) {
            ans ^= tree[r];
            r = F(r) - 1;
        }
        return ans;
    }

    // [l, r)
    ll query(ll l, ll r) {
        return query(r) ^ query(l);
```

```
    }

    void add(ll id, ll val) {
        while (id < n) {
            tree[id] ^= val;
            id = H(id);
        }
    }
};
```

5.6 HLD

```
struct HLD {
    ll n;
    vector<ll> top, pos, h, par;

    ll gpos = 0;

    SegmentTree st;

    void calc_sizes(ll v, vector<char>& used, const
    vector<vector<ll>>& g, vector<ll>& s) {
        s[v] = 1;
        used[v] = 1;

        for (ll u : g[v]) {
            if (!used[u]) {
                calc_sizes(u, used, g, s);
                s[v] += s[u];
            }
        }
    }

    void build_dec(ll v, ll p, ll pr, vector<char>&
    used, const vector<vector<ll>>& g, vector<ll>& s
    ) {
        used[v] = 1;
        pos[v] = gpos++;
        top[pos[v]] = pos[p];
        par[pos[v]] = pos[pr];
        if (pr != v)
            h[pos[v]] = h[pos[pr]] + 1;

        vector<ll> child;

        for (ll u : g[v]) {
            if (!used[u]) {
                child.push_back(u);
            }
        }

        for (int i = 1; i < child.size(); ++i) {
            if (s[child[i]] > s[child[0]]) {
                swap(child[i], child[0]);
            }
        }

        if (!child.empty()) {
            build_dec(child[0], p, v, used, g, s);
            for (int i = 1; i < child.size(); ++i) {
                build_dec(child[i], child[i], v,
    used, g, s);
            }
        }
    }

    HLD(ll N, const vector<vector<ll>>& g, const
    vector<ll>& a) {
        n = N;

        top.resize(n, 0);
        pos.resize(n, 0);
        h.resize(n, 0);
        par.resize(n, 0);
        vector<char> used(n, 0);
        vector<ll> s(n, 0);

        calc_sizes(0, used, g, s);
        used.assign(n, 0);
        build_dec(0, 0, 0, used, g, s);

        vector<ll> pa(n, 0);
        for (int i = 0; i < n; ++i) {
            pa[pos[i]] = a[i];
        }

        st = SegmentTree(pa);
    }

    ll query(ll u, ll v) {
        ll ans = 0;

        u = pos[u];
        v = pos[v];

        ll pu = top[u];
        ll pv = top[v];

        while (pu != pv) {
            if (h[pu] >= h[pv]) {
                ans += st.query(pu, u + 1).sum;
                u = par[pu];
                pu = top[u];
            } else {
                ans += st.query(pv, v + 1).sum;
                v = par[pv];
                pv = top[v];
            }
        }
        ans += st.query(min(u, v), max(u, v) + 1).
    sum;

        return ans;
    }

    void change(ll u, ll val) {
        u = pos[u];
        st.change(u, val);
    }
};
```

5.7 Implicit Segment Tree

```
struct ImplicitSegmentTree {
    struct Node {
        Node* left = nullptr;
        Node* right = nullptr;
        ll sum = 0;
        ll add = 0;
    };

    Node neutral = Node();
    ll n;
    Node* root = nullptr;

    Node merge(Node n1, Node n2) {
        Node res;
        res.sum = n1.sum + n2.sum;
        return res;
    }

    void fix(Node* node) {
        node->sum = 0;
        if (node->left) {
```

```
        node->sum += node->left->sum;
    }
    if (node->right) {
        node->sum += node->right->sum;
    }
}

void apply(Node* node, ll l, ll r, ll val) {
    node->sum += val * (r - l);
    node->add += val;
}

void push(Node* node, ll l, ll r) {
    if (node->add == 0) {
        return;
    }
    ll m = (l + r) / 2;
    if (!node->left) {
        node->left = new Node();
    }
    if (!node->right) {
        node->right = new Node();
    }
    apply(node->left, l, m, node->add);
    apply(node->right, m, r, node->add);
    node->add = 0;
}

ImplicitSegmentTree() {
}

ImplicitSegmentTree(ll N) {
    n = N;
}

ImplicitSegmentTree(const vector<ll>& start) {
    n = start.size();
    for (ll i = 0; i < n; i++) {
        change(i, start[i]);
    }
}

void change(Node*& node, ll l, ll r, ll id, ll
val) {
    if (!node) {
        node = new Node();
    }
    if (l + 1 == r) {
        node->sum = val;
        node->add = 0;
        return;
    }
    push(node, l, r);
    ll m = (l + r) / 2;
    if (id < m) {
        change(node->left, l, m, id, val);
    } else {
        change(node->right, m, r, id, val);
    }
    fix(node);
}

void change(ll id, ll val) {
    change(root, 0, n, id, val);
}

void add(Node*& node, ll l, ll r, ll ql, ll qr,
ll val) {
    if (qr <= l || r <= ql) {
        return;
    }
    if (!node) {
        node = new Node();
    }
    if (ql <= l && r <= qr) {
        apply(node, l, r, val);
        return;
    }
    push(node, l, r);
    ll m = (l + r) / 2;
    add(node->left, l, m, ql, qr, val);
    add(node->right, m, r, ql, qr, val);
    fix(node);
}

void add(ll ql, ll qr, ll val) {
    add(root, 0, n, ql, qr, val);
}

Node query(Node* node, ll l, ll r, ll ql, ll qr)
 {
    if (!node || qr <= l || r <= ql) {
        return neutral;
    }
    if (ql <= l && r <= qr) {
        return *node;
    }
    push(node, l, r);
    ll m = (l + r) / 2;
    return merge(query(node->left, l, m, ql, qr)
    , query(node->right, m, r, ql, qr));
    }

    Node query(ll ql, ll qr) {
        return query(root, 0, n, ql, qr);
    }
};
```

5.8 Persistent DSU

```
struct PersistentDSU {
    ll n;
    vector<ll> prev, size;

    PersistentDSU(ll N) : n(N) {
        prev.assign(n, -1);
        size.assign(n, 1);
    }

    ll root(ll a) {
        while (prev[a] != -1)
            a = prev[a];
        return a;
    }

    bool connected(ll a, ll b) {
        return root(a) == root(b);
    }

    vector<pair<ll, ll>> changes;  //

    bool unite(ll a, ll b) {
        a = root(a);
        b = root(b);
        if (a == b)
            return false;
        if (size[a] < size[b])
            swap(a, b);
        prev[b] = a;
        size[a] += size[b];
        changes.push_back({b, a});
        return true;
    }
```

```
    bool rollback() {
        if (changes.empty())
            return false;
        pair<ll, ll> ba = changes.back();
        changes.pop_back();
        prev[ba.first] = -1;
        size[ba.second] -= size[ba.first];
        return true;
    }
};
```

5.9 Persistent Segment Tree

```
struct PersistentSegmentTree {
    struct Node {
        int left = -1;
        int right = -1;
        ll val = inf;
        ll add = 0;
    };

    ll n;
    vector<Node> pool;
    vector<int> roots;

    PersistentSegmentTree() {
    }

    PersistentSegmentTree(const vector<ll>& start) {
        n = start.size();
        roots.push_back(build(0, n, start));
    }

    int new_node() {
        pool.push_back(Node());
        return pool.size() - 1;
    }

    int clone(int v) {
        pool.push_back(pool[v]);
        return pool.size() - 1;
    }

    void fix(int v) {
        pool[v].val = min(pool[pool[v].left].val,
    pool[pool[v].right].val);
    }

    void apply(int v, ll val) {
        pool[v].val += val;
        pool[v].add += val;
    }

    void push(int v) {
        if (pool[v].add == 0)
            return;
        pool[v].left = clone(pool[v].left);
        pool[v].right = clone(pool[v].right);
        apply(pool[v].left, pool[v].add);
        apply(pool[v].right, pool[v].add);
        pool[v].add = 0;
    }

    int build(ll l, ll r, const vector<ll>& start) {
        int v = new_node();
        if (l + 1 == r) {
            pool[v].val = start[l];
            return v;
        }
        ll m = (l + r) / 2;
        pool[v].left = build(l, m, start);
        pool[v].right = build(m, r, start);
        fix(v);
        return v;
    }

    ll query(int v, ll l, ll r, ll ql, ll qr, ll
    lazy) {
        if (r <= ql || qr <= l)
            return inf;
        if (ql <= l && r <= qr)
            return pool[v].val + lazy;

        lazy += pool[v].add;
        ll m = (l + r) / 2;
        return min(query(pool[v].left, l, m, ql, qr,
     lazy), query(pool[v].right, m, r, ql, qr, lazy)
    );
    }

    ll query(int t, ll ql, ll qr) {
        return query(roots[t], 0, n, ql, qr, 0);
    }

    void add(int v, ll l, ll r, ll ql, ll qr, ll val
    ) {
        if (r <= ql || qr <= l)
            return;
        if (ql <= l && r <= qr) {
            apply(v, val);
            return;
        }

        push(v);
        ll m = (l + r) / 2;
        if (ql < m) {
            pool[v].left = clone(pool[v].left);
            add(pool[v].left, l, m, ql, qr, val);
        }
        if (m < qr) {
            pool[v].right = clone(pool[v].right);
            add(pool[v].right, m, r, ql, qr, val);
        }
        fix(v);
    }

    void add(int t, ll ql, ll qr, ll val) {
        int root = clone(roots[t]);
        add(root, 0, n, ql, qr, val);
        roots.push_back(root);
    }
};
```

5.10 Segment Tree Beats

```
struct SegmentTreeBeats {
    struct Node {
        ll val = 0;
        ll firstMax = -inf;
        ll cntMax = 0;
        ll secondMax = -inf;

        ll min_eq = inf;

        Node(ll Val = 0, ll FirstMax = -inf, ll
     CntMax = 0, ll SecondMax = -inf) {
            val = Val;
            firstMax = FirstMax;
            cntMax = CntMax;
            secondMax = SecondMax;
        }
    };

    Node fromOne(ll a) {
```

```
    return Node(a, a, 1, -inf);
}

Node neutral = Node();
ll n;
vector<Node> tree;

SegmentTreeBeats(const vector<ll>& start) {
    n = start.size();
    tree.resize(4 * n + 3);
    init(start);
}

SegmentTreeBeats(ll N) {
    n = N;
    tree.resize(4 * n + 3);
    vector<ll> start(n, 0);
    init(start);
}

Node merge(Node n1, Node n2) {
    Node res(n1.val + n2.val);

    if (n1.firstMax > n2.firstMax) {
        res.firstMax = n1.firstMax;
        res.cntMax = n1.cntMax;
        res.secondMax = max(n1.secondMax, n2.
firstMax);
    } else if (n1.firstMax < n2.firstMax) {
        res.firstMax = n2.firstMax;
        res.cntMax = n2.cntMax;
        res.secondMax = max(n1.firstMax, n2.
secondMax);
    } else {
        res.firstMax = n1.firstMax;
        res.cntMax = n1.cntMax + n2.cntMax;
        res.secondMax = max(n1.secondMax, n2.
secondMax);
    }
    return res;
}

void fix(ll v, ll l, ll r) {
    tree[v] = merge(tree[v * 2 + 1], tree[v * 2
+ 2]);
}

void init(ll v, ll l, ll r, const vector<ll>&
start) {
    if (l + 1 == r) {
        tree[v] = fromOne(start[l]);
        return;
    }

    ll m = (r + l) / 2;
    init(v * 2 + 1, l, m, start);
    init(v * 2 + 2, m, r, start);
    fix(v, l, r);
}

void init(const vector<ll>& start) {
    init(0, 0, n, start);
}

// [l: r)
Node query(ll v, ll l, ll r, ll ql, ll qr) {
    if (ql <= l && r <= qr) {
        return tree[v];
    }
    if (r <= ql || qr <= l) {
        return neutral;
    }

    ll m = (r + l) / 2;
    push(v, l, r);
    return merge(query(v * 2 + 1, l, m, ql, qr),
 query(v * 2 + 2, m, r, ql, qr));
}

Node query(ll ql, ll qr) {
    return query(0, 0, n, ql, qr);
}

bool breakCondition(ll v, ll l, ll r, ll val) {
    return tree[v].firstMax <= val;
}

bool tagCondition(ll v, ll l, ll r, ll val) {
    return tree[v].secondMax < val;
}

void apply(ll v, ll l, ll r, ll val) {
    // only if tagCondition() == true
    ll diff = min(tree[v].firstMax, val) - tree[
v].firstMax;
    tree[v].val += tree[v].cntMax * diff;
    tree[v].firstMax += diff;

    tree[v].min_eq = min(val, tree[v].min_eq);
}

void push(ll v, ll l, ll r) {
    ll m = (r + l) / 2;

    apply(v * 2 + 1, l, m, tree[v].min_eq);
    apply(v * 2 + 2, m, r, tree[v].min_eq);
    tree[v].min_eq = inf;
}

void update(ll v, ll l, ll r, ll ql, ll qr, ll
val) {
    if (r <= ql || qr <= l || breakCondition(v,
l, r, val)) {
        return;
    }
    if (ql <= l && r <= qr && tagCondition(v, l,
 r, val)) {
        apply(v, l, r, val);
        return;
    }

    ll m = (r + l) / 2;
    push(v, l, r);
    update(v * 2 + 1, l, m, ql, qr, val);
    update(v * 2 + 2, m, r, ql, qr, val);
    fix(v, l, r);
}

void update(ll ql, ll qr, ll val) {
    update(0, 0, n, ql, qr, val);
}

pair<ll, ll> descent(ll v, ll l, ll r, ll ql, ll
 qr, ll x) {
    if (r <= ql || qr <= l)
        return {-1, x};
    if (ql <= l && r <= qr) {
        if (tree[v].val < x)
            return {-1, x - tree[v].val};
        if (l + 1 == r)
            return {l + 1, x};
    }
    if (l + 1 == r)
        return {-1, x};
```

```
        ll m = (r + l) / 2;
        push(v, l, r);
        auto left = descent(v * 2 + 1, l, m, ql, qr,
     x);
        if (left.first != -1)
            return left;
        return descent(v * 2 + 2, m, r, ql, qr, left
    .second);
    }

    ll descent(ll ql, ll qr, ll x) {
        if (x <= 0)
            return ql;
        return descent(0, 0, n, ql, qr, x).first;
    }
};
```

5.11 Treap

```
random_device rd;
mt19937 mersenne(rd());

struct Treap {
    Treap* left;
    Treap* right;
    ll x;
    unsigned int y;
    unsigned int size;
    ll dp;

    Treap(ll val = 0) {
        left = nullptr;
        right = nullptr;
        size = 1;
        x = val;
        y = mersenne();
        update(this);
    }

    friend ll get_dp(Treap* node) {
        if (node == nullptr)
            return 0;
        return node->dp;
    }

    friend unsigned int get_size(Treap* node) {
        if (node == nullptr)
            return 0;
        return node->size;
    }

    friend void update(Treap* node) {
        if (node == nullptr)
            return;
        node->size = 1 + get_size(node->left) +
    get_size(node->right);
        node->dp = max({node->x, get_dp(node->left),
     get_dp(node->right)});
    }

    // split by size
    friend pair<Treap*, Treap*> split_k(Treap* node,
     unsigned int k) {
        if (node == nullptr)
            return {nullptr, nullptr};
        if (get_size(node->left) < k) {
            auto res = split_k(node->right, k -
    get_size(node->left) - 1);
            node->right = res.first;
            update(node);
            return {node, res.second};
        } else {
            auto res = split_k(node->left, k);
            node->left = res.second;
            update(node);
            return {res.first, node};
        }
    }

    // split by dp: dp in left < k
    friend pair<Treap*, Treap*> split_dp(Treap* node
    , unsigned int k) {
        if (node == nullptr)
            return {nullptr, nullptr};
        if (get_dp(node) < k) {
            auto res = split_dp(node->right, k);
            node->right = res.first;
            update(node);
            return {node, res.second};
        } else {
            auto res = split_dp(node->left, k);
            node->left = res.second;
            update(node);
            return {res.first, node};
        }
    }

    friend Treap* merge(Treap* left, Treap* right) {
        if (left == nullptr)
            return right;
        if (right == nullptr)
            return left;
        if (left->y > right->y) {
            left->right = merge(left->right, right);
            update(left);
            return left;
        } else {
            right->left = merge(left, right->left);
            update(right);
            return right;
        }
    }

    friend Treap* insert(Treap* node, unsigned int
    pos, ll val) {
        auto res = split_k(node, pos);
        Treap* tr = new Treap(val);
        return merge(merge(res.first, tr), res.
    second);
    }

    friend Treap* erase(Treap* node, unsigned int
    pos) {
        auto res = split_k(node, pos + 1);
        auto left_res = split_k(res.first, pos);
        return merge(left_res.first, res.second);
    }

    // return {element, root}
    friend pair<Treap*, Treap*> kth_element(Treap*
    node, unsigned int k) {
        if (get_size(node) < k)
            return {nullptr, node};
        auto res = split_k(node, k + 1);
        auto left_res = split_k(res.first, k);
        return {left_res.second, merge(left_res.
    first, merge(left_res.second, res.second))};
    }

    // return sub segment [l, r) and remaineng Treap
     [0, l) + [r + 1, size)
    friend pair<Treap*, Treap*> cut(Treap* node,
    unsigned int l, unsigned int r) {
```

```
        auto res = split_k(node, r);
        auto left_res = split_k(res.first, l);
        return {left_res.second, merge(left_res.
    first, res.second)};
    }
};
```

6 Flows

6.1 Dinic

```
struct FlowSolver {
    struct DinicEdge {
        ll a, b, cap, flow;
    };

    vector<DinicEdge> edges;
    vector<vector<ll>> g;
    ll n;

    FlowSolver(ll N) {
        n = N;
        g.resize(n);
    }

    void add_edge(ll u, ll v, ll w) {
        g[u].push_back(edges.size());
        edges.push_back({u, v, w, 0});
        g[v].push_back(edges.size());
        edges.push_back({v, u, 0, 0});
    }

    bool bfs(ll s, ll t, vector<ll>& d) {
        queue<ll> q;
        d.assign(n, -1);
        d[s] = 0;
        q.push(s);

        while (!q.empty()) {
            ll v = q.front();
            q.pop();

            for (ll id : g[v]) {
                ll u = edges[id].b;
                if (d[u] == -1) {
                    if (edges[id].cap > edges[id].
    flow) {
                        d[u] = d[v] + 1;
                        q.push(u);
                    }
                }
            }
        }

        return d[t] != -1;
    }

    ll dfs(ll v, ll t, ll flow, vector<ll>& d,
    vector<ll>& ptr) {
        if (flow == 0) {
            return 0;
        }
        if (v == t) {
            return flow;
        }

        for (; ptr[v] < g[v].size(); ++ptr[v]) {
            ll id = g[v][ptr[v]];
            ll u = edges[id].b;

            if (d[u] == d[v] + 1) {
                ll add_flow = dfs(u, t, min(flow,
    edges[id].cap - edges[id].flow), d, ptr);
                if (add_flow > 0) {
                    edges[id].flow += add_flow;
                    edges[id ^ 1].flow -= add_flow;
                    return add_flow;
                }
            }
        }
        return 0;
    }

    ll dinic(ll s, ll t) {
        ll flow = 0;
        vector<ll> d;
        while (bfs(s, t, d)) {
            vector<ll> ptr(n, 0);
            while (ll add_flow = dfs(s, t, inf, d,
    ptr)) {
                flow += add_flow;
            }
        }
        return flow;
    }
};
```

6.2 Min Cost Max Flow Capacity Scaling

```
struct FlowSolver {
    struct Edge {
        ll a, b, cap, flow, cost;
    };

    vector<Edge> edges;
    vector<vector<ll>> g;
    vector<ll> e, pot;
    ll n;

    FlowSolver(ll N) {
        n = N;
        g.resize(n);
        e.assign(n, 0);
        pot.assign(n, 0);
    }

    void add_edge(ll u, ll v, ll w, ll c) {
        g[u].push_back(edges.size());
        edges.push_back({u, v, w, 0, c});
        g[v].push_back(edges.size());
        edges.push_back({v, u, 0, 0, -c});
    }

    ll dijkstra(vector<ll>& par, ll delta) {
        vector<ll> d(n, inf);
        par.assign(n, -1);
        priority_queue<pair<ll, ll>, vector<pair<ll,
     ll>>, greater<pair<ll, ll>>> q;
        for (int i = 0; i < n; ++i) {
            if (e[i] >= delta) {
                d[i] = 0;
                q.push({d[i], i});
            }
        }

        ll t = -1;

        while (!q.empty()) {
            auto [dv, v] = q.top();
            q.pop();

            if (dv != d[v]) {
                continue;
            }
```

```
        if (e[v] <= -delta) {
            t = v;
            break;
        }

        for (ll id : g[v]) {
            if (edges[id].cap - edges[id].flow <
 delta) {
                continue;
            }
            ll u = edges[id].b;
            ll ncost = dv + edges[id].cost + pot
[v] - pot[u];
            if (d[u] > ncost) {
                d[u] = ncost;
                q.push({d[u], u});
                par[u] = id;
            }
        }
    }
    if (t == -1) {
        return -1;
    }
    for (int i = 0; i < n; ++i) {
        pot[i] += min(d[i], d[t]);
    }

    return t;
}

ll push_flow(ll t, const vector<ll>& par, ll
delta) {
    ll v = t;
    ll cost = 0;
    while (par[v] != -1) {
        ll id = par[v];
        edges[id].flow += delta;
        edges[id ^ 1].flow -= delta;
        v = edges[par[v]].a;
        cost += delta * edges[id].cost;
    }
    e[t] += delta;
    e[v] -= delta;

    return cost;
}

ll delta_iter(ll delta) {
    ll cost = 0;
    // maxing negative edges
    for (ll id = 0; id < edges.size(); ++id) {
        if (edges[id].cap - edges[id].flow <
delta) {
            continue;
        }

        if (edges[id].cost + pot[edges[id].a] -
pot[edges[id].b] < 0) {
            ll dflow = edges[id].cap - edges[id
].flow;
            cost += dflow * edges[id].cost;
            edges[id].flow += dflow;
            edges[id ^ 1].flow -= dflow;
            e[edges[id].a] -= dflow;
            e[edges[id].b] += dflow;
        }
    }

    vector<ll> par;
    ll t = -1;
    while ((t = dijkstra(par, delta)) != -1) {
        cost += push_flow(t, par, delta);
    }

    return cost;
}

ll cap_scaling(ll s, ll t, ll F) {
    ll max_w = 0;
    for (const auto& edge : edges) {
        max_w = max(max_w, edge.cap - edge.flow)
;
    }
    ll delta = 1;
    while ((delta << 1) <= max_w) {
        delta <<= 1;
    }
    e[s] = F;
    e[t] = -F;

    ll cost = 0;
    while (delta >= 1) {
        cost += delta_iter(delta);
        delta >>= 1;
    }
    return cost;
}
};
```

6.3 Min Cost Max Flow

```
struct FlowSolver {
    struct Edge {
        ll a, b, cap, flow, cost;
    };

    vector<Edge> edges;
    vector<vector<ll>> g;
    ll n;

    FlowSolver(ll N) {
        n = N;
        g.resize(n);
    }

    void add_edge(ll u, ll v, ll w, ll c) {
        g[u].push_back(edges.size());
        edges.push_back({u, v, w, 0, c});
        g[v].push_back(edges.size());
        edges.push_back({v, u, 0, 0, -c});
    }

    void ford_bellman(ll s, vector<ll>& d) {
        d.assign(n, 0);

        bool change = true;
        while (change) {
            change = false;

            for (ll id = 0; id < edges.size(); ++id)
 {
                if (edges[id].cap == edges[id].flow)
 {
                    continue;
                }

                ll v = edges[id].a;
                ll u = edges[id].b;
                if (d[u] > d[v] + edges[id].cost) {
                    change = true;
                    d[u] = d[v] + edges[id].cost;
                }
            }
```

```
    }
}

bool dijkstra(ll s, ll t, vector<ll>& d, vector<
ll>& par) {
    vector<ll> nd(n, inf);
    par.assign(n, -1);
    priority_queue<pair<ll, ll>, vector<pair<ll,
 ll>>, greater<pair<ll, ll>>> q;
    nd[s] = 0;
    q.push({nd[s], s});

    while (!q.empty()) {
        auto [ndv, v] = q.top();
        q.pop();

        if (ndv != nd[v]) {
            continue;
        }

        for (ll id : g[v]) {
            if (edges[id].cap == edges[id].flow)
 {
                continue;
            }
            ll u = edges[id].b;
            ll ncost = ndv + edges[id].cost + d[
v] - d[u];
            if (nd[u] > ncost) {
                nd[u] = ncost;
                q.push({nd[u], u});
                par[u] = id;
            }
        }
    }
    for (int i = 0; i < n; ++i) {
        if (nd[i] != inf) {
            d[i] += nd[i];
        }
    }

    return nd[t] != inf;
}

pair<ll, ll> push_flow(ll s, ll t, const vector<
ll>& par) {
    vector<ll> path;
    ll add_flow = inf;
        ll v = t;
        while (v != s) {
            path.push_back(par[v]);
            add_flow = min(add_flow, edges[par[v]].
    cap - edges[par[v]].flow);
            v = edges[par[v]].a;
        }
        ll cost = 0;
        for (ll id : path) {
            cost += add_flow * edges[id].cost;
            edges[id].flow += add_flow;
            edges[id ^ 1].flow -= add_flow;
        }
        return {add_flow, cost};
    }

    pair<ll, ll> ssp(ll s, ll t) {
        ll flow = 0, cost = 0;
        vector<ll> d, par;
        ford_bellman(s, d);
        while (dijkstra(s, t, d, par)) {
            auto [add_flow, add_cost] = push_flow(s,
     t, par);
            flow += add_flow;
            cost += add_cost;
        }
        return {flow, cost};
    }
};
```

7 Geometry

7.1 Basic

```
bool point_on_line(const Point& p, const Line& line)
     {
    return sign_vectp(p - line.p1, line.p2 - line.p1
    ) == 0;
}

bool point_on_segment(const Point& p, const Segment&
     seg) {
    return sign_vectp(p - seg.p2, seg.p1 - seg.p2)
    == 0 &&
           sign_scalp(p - seg.p2, seg.p1 - seg.p2)
    >= 0 &&
           sign_scalp(p - seg.p1, seg.p2 - seg.p1)
    >= 0;
}

bool is_segment_intersects_line(const Segment& seg,
    const Line& line) {
    return sign_vectp(seg.p1 - line.p1, line.p2 -
    line.p1) *
                sign_vectp(seg.p2 - line.p1, line.p2
    - line.p1) <=
           0;
}

bool is_segment_intersects_segment(const Segment&
    seg1, const Segment& seg2) {
    if (sign_vectp(seg1.p2 - seg1.p1, seg2.p2 - seg2
    .p1) == 0) {
        return point_on_segment(seg1.p1, seg2) ||
    point_on_segment(seg1.p2, seg2) ||
                point_on_segment(seg2.p1, seg1) ||
    point_on_segment(seg2.p2, seg1);
    }
    return is_segment_intersects_line(seg1, Line(
    seg2)) &&
           is_segment_intersects_line(seg2, Line(
    seg1));
}

// intersection of supporting lines; segments must
    be non-parallel
Point lines_intersection(const Line& a, const Line&
    b) {
    Point A = a.p1, B = a.p2, C = b.p1, D = b.p2;
    ld t = vectp(C - A, D - C) / vectp(B - A, D - C)
    ;
    return A + (B - A) * t;
}

// unique intersection point; call only if segments
    properly intersect (non-collinear)
Point segments_intersection(const Segment& a, const
    Segment& b) {
    return lines_intersection(Line(a), Line(b));
}

// foot of perpendicular from p to line
Point project(const Point& p, const Line& line) {
    Point d = line.p2 - line.p1;
    return line.p1 + d * (scalp(p - line.p1, d) / d.
    abs2());
```

```
}

// reflection of p over line
Point reflect(const Point& p, const Line& line) {
    return project(p, line) * 2 - p;
}
```

7.2 Halfplanes Intersection

```
// Closed half-plane to the left of p1 -> p2; p1 != p2.
struct Halfplane {
    Point p, d;
    ld angle;

    Halfplane(const Line& line) : p(line.p1), d(line.p2 - line.p1) {
        d = d / d.abs();
        if (d.y == 0) d.y = 0; // Canonicalize signed zero at the angle cut.
        angle = d.ang();
    }

    bool outside(const Point& q) const {
        return vectp(d, q - p) < -EPS;
    }

    friend Point intersection(const Halfplane& a, const Halfplane& b) {
        return a.p + a.d * (vectp(b.p - a.p, b.d) / vectp(a.d, b.d));
    }
};

// O(n log n) time, O(n) memory. Uses Point, Line, EPS from Basic.h.
// Returns CCW vertices of intersection with [-bound, bound]^2; bound > 0.
// Choose bound to contain the desired answer; unbounded regions are clipped.
// Returns {} if empty, {p} for a point, {a, b} for a segment.
// Vertices within EPS are considered equal; no area threshold is used.
// Floating point: adjust EPS / rescale coordinates for the problem.
vector<Point> halfplanes_intersection(const vector<Line>& lines, ld bound) {
    vector<Halfplane> h;
    for (const Line& line : lines) {
        h.emplace_back(line);
    }
    vector<Point> box = {{-bound, -bound}, {bound, -bound},
                         {bound, bound}, {-bound, bound}};
    for (int i = 0; i < 4; ++i) {
        h.emplace_back(Line(box[i], box[(i + 1) % 4]));
    }
    // Exact comparison: using EPS in a sort comparator is invalid.
    sort(h.begin(), h.end(), [](const Halfplane& a, const Halfplane& b) {
        return a.angle < b.angle;
    });

    // If opposite constraints force a line, clip its parameter by ALL sides.
    // Called at most once, followed by return: O(n), including degeneracies.
    auto on_line = [&](const Halfplane& a) -> vector<Point> {
        ld lo = -numeric_limits<ld>::infinity(), hi = -lo;
        for (const Halfplane& b : h) {
            ld k = vectp(b.d, a.d), c = vectp(b.d, a.p - b.p);
            if (fabsl(k) <= EPS) {
                if (c < -EPS) return {};
            } else if (k > 0) {
                lo = max(lo, -c / k);
            } else {
                hi = min(hi, -c / k);
            }
        }
        if (lo > hi + EPS) return {};
        if (hi - lo <= EPS) return {a.p + a.d * ((lo + hi) / 2)};
        return {a.p + a.d * lo, a.p + a.d * hi};
    };

    deque<Halfplane> q;
    for (const Halfplane& cur : h) {
        while (q.size() > 1 && cur.outside(intersection(q[q.size() - 2], q.back()))) {
            q.pop_back();
        }
        while (q.size() > 1 && cur.outside(intersection(q[0], q[1]))) {
            q.pop_front();
        }
        if (!q.empty() && fabsl(vectp(q.back().d, cur.d)) <= EPS) {
            if (scalp(q.back().d, cur.d) < 0) {
                if (fabsl(vectp(q.back().d, cur.p - q.back().p)) <= EPS) {
                    return on_line(q.back());
                }
                return {}; // With the bounding box, a feasible strip cannot occur here.
            }
            if (cur.outside(q.back().p)) {
                q.pop_back(); // Keep the more restrictive parallel half-plane.
            } else {
                continue;
            }
        }
        q.push_back(cur);
    }
    while (q.size() > 2 && q.front().outside(intersection(q[q.size() - 2], q.back()))) {
        q.pop_back();
    }
    while (q.size() > 2 && q.back().outside(intersection(q[0], q[1]))) {
        q.pop_front();
    }
    if (q.size() < 3) return {};
    if (fabsl(vectp(q.back().d, q.front().d)) <= EPS) {
        if (scalp(q.back().d, q.front().d) < 0 &&
            fabsl(vectp(q.back().d, q.front().p - q.back().p)) <= EPS) {
            return on_line(q.back());
        }
        return {};
    }
    vector<Point> res;
    for (int i = 0; i < q.size(); ++i) {
        Point p = intersection(q[i], q[(i + 1) % q.size()]);
        if (res.empty() || (p - res.back()).abs() >
```

```
    EPS) res.push_back(p);
    }
    if (res.size() > 1 && (res.front() - res.back())
    .abs() <= EPS) res.pop_back();
    // A collapsed deque may describe a point
    excluded by discarded sides.
    // Check the point / both segment endpoints
    against all original constraints.
    if (res.size() <= 2) {
        for (const Point& p : res) {
            for (const Halfplane& a : h) {
                if (a.outside(p)) return {};
            }
        }
    }
    return res;
}
```

8 Graphs

8.1 2 SAT

```
struct SATSolver {
    ll n;
    vector<vector<ll>> g, gt;
    vector<char> used;
    vector<ll> order, comp;

    void init(ll N) {
        n = N;
        g.assign(2 * n, {});
        gt.assign(2 * n, {});
    }

    // literal: var in [0..n), val=false -> 2*var,
    val=true -> 2*var+1
    ll lit(ll var, ll val) {
        return 2 * var + (val ? 1 : 0);
    }
    ll neg(ll x) {
        return x ^ 1;
    }

    void add_imp(ll a, ll b) {
        g[a].push_back(b);
        gt[b].push_back(a);
    }

    // (a_var is a_val) OR (b_var is b_val)
    void add_or(ll a_var, ll a_val, ll b_var, ll
    b_val) {
        ll a = lit(a_var, a_val);
        ll b = lit(b_var, b_val);
        add_imp(neg(a), b);
        add_imp(neg(b), a);
    }

    void dfs1(ll v) {
        used[v] = 1;
        for (ll to : g[v])
            if (!used[to])
                dfs1(to);
        order.push_back(v);
    }

    void dfs2(ll v, ll cl) {
        comp[v] = cl;
        for (ll to : gt[v])
            if (comp[to] == -1)
                dfs2(to, cl);
    }

    // returns empty vector if unsat; otherwise
    assignment[i] in {0,1}
    vector<ll> solve() {
        used.assign(2 * n, 0);
        order.clear();
        for (ll i = 0; i < 2 * n; ++i)
            if (!used[i])
                dfs1(i);

        comp.assign(2 * n, -1);
        ll j = 0;
        for (ll i = 2 * n - 1; i >= 0; --i) {
            ll v = order[i];
            if (comp[v] == -1)
                dfs2(v, j++);
        }

        vector<ll> ans(n);
        for (ll i = 0; i < n; ++i) {
            if (comp[2 * i] == comp[2 * i + 1])
                return {};
            ans[i] = (comp[2 * i] < comp[2 * i + 1])
;
        }
        return ans;
    }
};
```

8.2 Euler Path

```
void cycle(ll prev, ll v, vector<vector<pair<ll, ll
    >>>& g, vector<ll>& ans) {
    while (!g[v].empty()) {
        auto iu = g[v].back();
        ll u = iu.second;
        g[v].pop_back();
        cycle(iu.first, u, g, ans);
    }
    if (prev >= 0)
        ans.push_back(prev);
}
```

8.3 Hopcroft Karp

```
struct HopcroftKarp {
    ll n1, n2;
    vector<vector<ll>> g;
    vector<ll> dist;
    vector<ll> mt1, mt2;

    void init(ll N1, ll N2) {
        n1 = N1;
        n2 = N2;
        g.assign(n1, {});
        mt1.assign(n1, -1);
        mt2.assign(n2, -1);
        dist.assign(n1, 0);
    }

    void add_edge(ll u, ll v) {
        // u in [0..n1), v in [0..n2)
        g[u].push_back(v);
    }

    bool bfs() {
        queue<ll> q;
        for (ll i = 0; i < n1; ++i) {
            if (mt1[i] == -1) {
                dist[i] = 0;
                q.push(i);
            } else {
                dist[i] = -1;
```

```
        }
    }

    bool found = false;
    while (!q.empty()) {
        ll v = q.front();
        q.pop();

        for (ll to : g[v]) {
            ll u = mt2[to];
            if (u == -1) {
                found = true;
            } else if (dist[u] == -1) {
                dist[u] = dist[v] + 1;
                q.push(u);
            }
        }
    }
    return found;
}

bool dfs(ll v) {
    for (ll to : g[v]) {
        ll u = mt2[to];
        if (u == -1 || (dist[u] == dist[v] + 1
&& dfs(u))) {
            mt1[v] = to;
            mt2[to] = v;
            return true;
        }
    }
    dist[v] = -1;
    return false;
}

ll solve() {
    ll matching = 0;
    while (bfs()) {
        for (ll i = 0; i < n1; ++i) {
            if (mt1[i] == -1) {
                if (dfs(i)) {
                    ++matching;
                }
            }
        }
    }
    return matching;
}
```

8.4 Hungarian

```
};

// a size is (n + 1) x (m + 1), 1-indexing
// n <= m; if n < m than a[i][j] must be >= 0
pair<ll, vector<int>> hungarian(const vector<vector<
    ll>>& a, int n, int m) {
    vector<ll> u(n + 1, 0), v(m + 1, 0);
    vector<int> p(m + 1, 0), way(m + 1, 0);
    for (int i = 1; i <= n; ++i) {
        int j0 = 0;
        p[j0] = i;

        vector<char> used(m + 1, 0);
        vector<ll> minv(m + 1, inf);
        do {
            used[j0] = true;
            int i0 = p[j0];
            int j1 = -1, delta = inf;

            // run step of dfs from i0
            for (int j = 1; j <= m; ++j) {
                if (used[j]) {
                    continue;
                }

                int cur = a[i0][j] - u[i0] - v[j];
                if (cur < minv[j]) {
                    minv[j] = cur;
                    way[j] = j0;
                }
                if (minv[j] < delta) {
                    delta = minv[j];
                    j1 = j;
                }
            }

            // update potentials
            for (int j = 0; j <= m; ++j) {
                if (used[j]) {
                    u[p[j]] += delta;
                    v[j] -= delta;
                } else {
                    minv[j] -= delta;
                }
            }

            j0 = j1;
        } while (p[j0] != 0);

        while (j0) {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        }
    }

    return {-v[0], p};
}
```

8.5 Initial Pairing

```
int init_pairing(int n1, int n2, const vector<vector
    <int>>& g, vector<int>& mt, vector<int>& rmt) {
    auto PACK = [](int x, int y) { return ((ll)x <<
    30) + y; };
    auto SUNPACK = [](int x) {
        const static ll mask = (1ll << 30) - 1;
        return x & mask;
    };

    vector<int> deg(n1, 0);
    vector<vector<int>> ng(n1);
    vector<vector<pair<int, int>>> rg(n2);
    vector<int> next(n1, 0);
    set<ll> que;
    for (int i = 0; i < n1; ++i) {
        for (auto j : g[i]) {
            ++deg[i];
            rg[j].push_back({i, ng[i].size()});
            ng[i].push_back(j);
        }
        que.insert(PACK(deg[i], i));
    }

    int pairing = 0;

    while (!que.empty()) {
        auto res = *que.begin();
        que.erase(res);

        int i = SUNPACK(res);
        if (deg[i] == 0) {
            continue;
        }
```

```
        while (ng[i][next[i]] == -1) {
            ++next[i];
        }
        int j = ng[i][next[i]++];

        deg[i] = 0;
        for (auto ks : rg[j]) {
            int k = ks.first;
            int sz = ks.second;
            if (deg[k] != 0) {
                ng[k][sz] = -1;
                que.erase(PACK(deg[k], k));
                --deg[k];
                que.insert(PACK(deg[k], k));
            }
        }
        mt[j] = i;
        rmt[i] = j;
        ++pairing;
    }
    return pairing;
}

vector<int> fast_kuhn(int n1, int n2, vector<vector<
    int>>& g) {
    vector<char> used(n1, 0);
    vector<int> mt(n2, -1), rmt(n1, -1);

    init_pairing(n1, n2, g, mt, rmt);

    int mark = 0;

    for (int run = 1; run;) {
        ++mark;
        run = 0;
        for (int i = 0; i < n1; ++i) {
            if (rmt[i] == -1 && kuhn_dfs(i, used,
    mark, g, mt)) {
                run = 1;
            }
        }

        for (int j = 0; j < n2; ++j) {
            if (mt[j] != -1) {
                rmt[mt[j]] = j;
            }
        }
    }

    return mt;
}
```

8.6 Kuhn

```
bool kuhn_dfs(int v, vector<int>& used, int mark,
    vector<vector<int>>& g, vector<int>& mt) {
    if (used[v] == mark) {
        return false;
    }
    used[v] = mark;

    for (int u : g[v]) {
        if (mt[u] == -1 || kuhn_dfs(mt[u], used,
    mark, g, mt)) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}

vector<int> kuhn(int n1, int n2, vector<vector<int
    >>& g) {
    vector<int> used(n1, 0);
    vector<int> mt(n2, -1);

    int mark = 1;

    for (int i = 0; i < n1; ++i) {
        if (kuhn_dfs(i, used, mark, g, mt)) {
            ++mark;
        }
    }

    return mt;
}
```

9 Matroids

9.1 MatroidExamples

```
struct Matroid {
    virtual int size() const = 0;
    virtual void build(const vector<int> &on) = 0;
    virtual vector<int> circuit(int u) const = 0;
    virtual ~Matroid() = default;
};

// independent  <=>  forest
struct ForestMatroid : Matroid {
    int n;
    vector<pair<int, int>> edges;
    ForestMatroid() {
    }
    ForestMatroid(int n_) : n(n_) {
    }
    void ae(int u, int v) {
        edges.emplace_back(u, v);
    }
    inline int size() const override {
        return edges.size();
    }
    void build(const vector<int> &on) override {
        const int m = edges.size();
        g.assign(n, {});
        for (int i = 0; i < m; ++i) {
            if (on[i]) {
                g[edges[i].first].push_back(i);
                g[edges[i].second].push_back(i);
            }
        }
        rs.assign(n, -1);
        pari.assign(n, -1);
        dep.assign(n, -1);
        for (int r = 0; r < n; ++r)
            if (!~rs[r])
                dfs(r, r, -1, 0);
    }
    // i0, then tree edges from edges[i0].first to
    edges[i0].second
    vector<int> circuit(int i0) const override {
        int u = edges[i0].first, v = edges[i0].
    second;
        if (rs[u] != rs[v])
            return {};
        vector<int> isU, isV;
        auto upU = [&]() -> void {
            const int i = pari[u];
            isU.push_back(i);
            u ^= edges[i].first ^ edges[i].second;
        };
        auto upV = [&]() -> void {
```

```
            const int i = pari[v];
            isV.push_back(i);
            v ^= edges[i].first ^ edges[i].second;
        };
        for (; dep[u] > dep[v]; upU()) {
        }
        for (; dep[u] < dep[v]; upV()) {
        }
        for (; u != v; upU(), upV()) {
        }
        reverse(isV.begin(), isV.end());
        vector<int> is{i0};
        is.insert(is.end(), isU.begin(), isU.end());
        is.insert(is.end(), isV.begin(), isV.end());
        return is;
    }
    vector<vector<int>> g;
    vector<int> rs, pari, dep;
    void dfs(int r, int u, int pi, int d) {
        rs[u] = r;
        pari[u] = pi;
        dep[u] = d;
        for (const int i : g[u])
            if (pi != i)
                dfs(r, edges[i].first ^ edges[i].
    second ^ u, i, d + 1);
    }
};

// independent <=> each connected component
    contains (<= 1) cycle
struct PseudoforestMatroid : Matroid {
    int n;
    vector<pair<int, int>> edges;
    PseudoforestMatroid() {
    }
    PseudoforestMatroid(int n_) : n(n_) {
    }
    void ae(int u, int v) {
        edges.emplace_back(u, v);
    }
    inline int size() const override {
        return edges.size();
    }
    void build(const vector<int> &on) override {
        const int m = edges.size();
        g.assign(n, {});
        for (int i = 0; i < m; ++i) {
            if (on[i]) {
                g[edges[i].first].push_back(i);
                g[edges[i].second].push_back(i);
            }
        }
        rs.assign(n, -1);
        pari.assign(n, -1);
        dep.assign(n, -1);
        cycs.assign(n, -1);
        for (int r = 0; r < n; ++r)
            if (!~rs[r])
                dfs(r, r, -1, 0);
    }
    vector<int> circuit(int i0) const override {
        vector<int> is;
        // Add edges in the smallest subtree
    containing us.
        auto addSubtree = [&](vector<int> us) ->
    void {
            while (true) {
                sort(us.begin(), us.end());
                us.erase(unique(us.begin(), us.end()
    ), us.end());
                if (us.size() == 1)
                    break;
                int mx = -1;
                for (const int u : us)
                    if (mx < dep[u])
                        mx = dep[u];
                for (int &u : us)
                    if (mx == dep[u]) {
                        const int i = pari[u];
                        is.push_back(i);
                        u ^= edges[i].first ^ edges[
    i].second;
                    }
            }
        };
        const int u = edges[i0].first, v = edges[i0
    ].second;
        if (rs[u] != rs[v]) {
            if (~cycs[rs[u]] && ~cycs[rs[v]]) {
                is.push_back(i0);
                is.push_back(cycs[rs[u]]);
                addSubtree({u, edges[cycs[rs[u]]].
    first, edges[cycs[rs[u]]].second});
                is.push_back(cycs[rs[v]]);
                addSubtree({v, edges[cycs[rs[v]]].
    first, edges[cycs[rs[v]]].second});
            }
        } else {
            if (~cycs[rs[u]]) {
                is.push_back(i0);
                is.push_back(cycs[rs[u]]);
                addSubtree({u, v, edges[cycs[rs[u
    ]]].first, edges[cycs[rs[u]]].second});
            }
        }
        return is;
    }
    vector<vector<int>> g;
    vector<int> rs, pari, dep, cycs;
    void dfs(int r, int u, int pi, int d) {
        rs[u] = r;
        pari[u] = pi;
        dep[u] = d;
        for (const int i : g[u]) {
            if (pi != i) {
                const int v = edges[i].first ^ edges
    [i].second ^ u;
                if (~rs[v]) {
                    cycs[r] = i;
                } else {
                    dfs(r, v, i, d + 1);
                }
            }
        }
    }
};

// independent <=> #{ i | colors[i] = c } <= lims[
    c]
struct PartitionMatroid : Matroid {
    int n;
    vector<int> colors;
    vector<int> lims;
    PartitionMatroid() {
    }
    PartitionMatroid(const vector<int> &colors_,
    const vector<int> &lims_)
        : n(colors_.size()), colors(colors_), lims(
    lims_) {
    }
    inline int size() const override {
        return n;
    }
```

```
    void build(const vector<int> &on) override {
        iss.assign(lims.size(), {});
        for (int i = 0; i < n; ++i)
            if (on[i])
                iss[colors[i]].push_back(i);
    }
    vector<int> circuit(int i0) const override {
        const int c = colors[i0];
        if (iss[c].size() == lims[c]) {
            vector<int> is = iss[c];
            is.push_back(i0);
            return is;
        } else {
            return {};
        }
    }
    vector<vector<int>> iss;
};

// independent <=> vectors are linearly
    independent over GF(2)
struct LinearMatroid : Matroid {
    int n;
    int maxLog;
    vector<ull> vals;

    int blocks;
    vector<ull> basis;
    vector<vector<ull>> basisMask;

    LinearMatroid() {
    }
    LinearMatroid(const vector<ull> &vals_, int
    maxLog_ = 64) {
        vals = vals_;
        n = vals.size();
        maxLog = maxLog_;
        blocks = (n + 63) / 64;
    }

    inline int size() const override {
        return n;
    }

    void set_bit(vector<ull> &mask, int i) {
        mask[i >> 6] |= (1ULL << (i & 63));
    }

    void xor_mask(vector<ull> &a, const vector<ull>
    &b) {
        for (int i = 0; i < blocks; ++i)
            a[i] ^= b[i];
    }

    void add_vector(ull x, int id) {
        vector<ull> cur(blocks, 0);
        set_bit(cur, id);

        for (int b = maxLog - 1; b >= 0; --b) {
            if (((x >> b) & 1ULL) == 0)
                continue;
            if (!basis[b]) {
                basis[b] = x;
                basisMask[b] = cur;
                return;
            }
            x ^= basis[b];
            xor_mask(cur, basisMask[b]);
        }
    }

    void build(const vector<int> &on) override {
        basis.assign(maxLog, 0);
        basisMask.assign(maxLog, vector<ull>(blocks,
     0));
        for (int i = 0; i < n; ++i) {
            if (on[i])
                add_vector(vals[i], i);
        }
    }

    vector<int> circuit(int u) const override {
        ull x = vals[u];
        vector<ull> cur(blocks, 0);
        cur[u >> 6] |= (1ULL << (u & 63));

        for (int b = maxLog - 1; b >= 0; --b) {
            if (((x >> b) & 1ULL) == 0)
                continue;
            if (!basis[b]) {
                return {};
            }
            x ^= basis[b];
            for (int i = 0; i < blocks; ++i)
                cur[i] ^= basisMask[b][i];
        }
        vector<int> res;
        for (int i = 0; i < n; ++i) {
            if ((cur[i >> 6] >> (i & 63)) & 1ULL)
                res.push_back(i);
        }
        return res;
    }
};

// independent <=> selected left vertices can be
    fully matched to right part
struct TransversalMatroid : Matroid {
    int n, m;
    vector<vector<int>> g;
    vector<int> cur_on;

    vector<int> matchL, matchR;
    vector<int> used;
    int used_timer;

    TransversalMatroid() {
    }
    TransversalMatroid(int n_, int m_) : n(n_), m(m_
    ) {
        g.assign(n, {});
    }

    void ae(int u, int v) {
        g[u].push_back(v);
    }

    inline int size() const override {
        return n;
    }

    bool dfs_kuhn(int u) {
        if (used[u] == used_timer)
            return false;
        used[u] = used_timer;
        for (int v : g[u]) {
            int w = matchR[v];
            if (w == -1 || dfs_kuhn(w)) {
                matchL[u] = v;
                matchR[v] = u;
                return true;
            }
        }
```

```
        return false;
    }

    void build(const vector<int> &on) override {
        cur_on = on;
        matchL.assign(n, -1);
        matchR.assign(m, -1);
        used.assign(n, 0);
        used_timer = 0;
        for (int u = 0; u < n; ++u) {
            if (cur_on[u]) {
                ++used_timer;
                dfs_kuhn(u);
            }
        }
    }

    vector<int> circuit(int u0) const override {
        vector<int> visL(n, 0), visR(m, 0);
        queue<int> q;
        visL[u0] = 1;
        q.push(u0);

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) {
                if (visR[v])
                    continue;
                visR[v] = 1;
                if (matchR[v] == -1) {
                    return {};
                }
                int w = matchR[v];
                if (!visL[w] && cur_on[w]) {
                    visL[w] = 1;
                    q.push(w);
                }
            }
        }

        vector<int> res;
        for (int u = 0; u < n; ++u) {
            if (u == u0 || (cur_on[u] && visL[u]))
                res.push_back(u);
        }
        return res;
    }
};
```

9.2 MatroidIntersectionUnweighted

```
// Find max size independet set.
//   score: max size
//   on: optimal independent set (0/1)
// M0, M1: matroid
//   size(): |ground set|
//   build(on): Current independent set is given.
//   circuit(u): Returns the circuit by adding u (
    must contain u), or {} if still independent.
pair<int, vector<int>> matroidIntersection(Matroid&
    m0, Matroid& m1) {
    const int n = m0.size();
    vector<int> on(n, 0);
    while (true) {
        m0.build(on);
        m1.build(on);
        vector<vector<int>> graph(n + 2);
        for (int u = 0; u < n; ++u) {
            if (!on[u]) {
                const vector<int> c0 = m0.circuit(u)
    ;
                if (c0.empty())
                    graph[n].push_back(u);
                for (const int v : c0)
                    if (u != v)
                        graph[v].push_back(u);
                const vector<int> c1 = m1.circuit(u)
    ;
                if (c1.empty())
                    graph[u].push_back(n + 1);
                for (const int v : c1)
                    if (u != v)
                        graph[u].push_back(v);
            }
        }
        queue<int> que;
        vector<int> prv(n + 2, -1);
        prv[n] = -2;
        que.push(n);
        while (!que.empty() && !~prv[n + 1]) {
            const int u = que.front();
            que.pop();
            for (const int v : graph[u]) {
                if (!~prv[v]) {
                    prv[v] = u;
                    que.push(v);
                }
            }
        }
        if (~prv[n + 1]) {
            for (int u = n + 1; (u = prv[u]) != n;)
                on[u] ^= 1;
        } else {
            break;
        }
    }
    int score = 0;
    for (int u = 0; u < n; ++u)
        if (on[u])
            ++score;
    return make_pair(score, on);
};
```

9.3 MatroidIntersectionWeighted

```
// Find max weight independet set.
//   score: max weight
//   on: optimal independent set (0/1)
// M0, M1: matroid
//   size(): |ground set|
//   build(on): Current independent set is given.
//   circuit(u): Returns the circuit by adding u (
    must contain u), or {} if still independent.
pair<ll, vector<int>> matroidIntersection(Matroid&
    m0, Matroid& m1, const vector<ll>& weights) {
    const int n = weights.size();
    vector<int> on(n, 0);
    while (true) {
        m0.build(on);
        m1.build(on);
        vector<vector<int>> graph(n + 2);
        for (int u = 0; u < n; ++u) {
            if (!on[u]) {
                const vector<int> c0 = m0.circuit(u)
    ;
                if (c0.empty())
                    graph[n].push_back(u);
                for (const int v : c0)
                    if (u != v)
                        graph[v].push_back(u);
                const vector<int> c1 = m1.circuit(u)
    ;
                if (c1.empty())
```

```
                graph[u].push_back(n + 1);
            for (const int v : c1)
                if (u != v)
                    graph[u].push_back(v);
        }
    }
    queue<int> que;
    vector<int> inq(n + 2, 0);
    vector<int> prv(n + 2, -1);
    vector<ll> dist(n + 2, 0);
    prv[n] = -2;
    inq[n] = 1;
    que.push(n);
    while (!que.empty()) {
        const int u = que.front();
        que.pop();
        inq[u] = 0;
        for (const int v : graph[u]) {
            ll cc = dist[u];
            if (v < n)
                on[v] ? (cc -= weights[v]) : (cc
 += weights[v]);
            if (!~prv[v] || dist[v] < cc) {
                prv[v] = u;
                dist[v] = cc;
                if (!inq[v]) {
                    inq[v] = 1;
                    que.push(v);
                }
            }
        }
    }
    if (~prv[n + 1]) {
        for (int u = n + 1; (u = prv[u]) != n;)
            on[u] ^= 1;
    } else {
        break;
    }
  }
  ll score = 0;
  for (int u = 0; u < n; ++u)
    if (on[u])
      score += weights[u];
  return make_pair(score, on);
};
```

10 Number Theory

10.1 CRT

```
ll inv(ll a, ll m) {
    return a == 1 ? 1 : (1 - inv(m % a, a) * m) / a
    + m;
}

ll crt(ll a1, ll m1, ll a2, ll m2) {
    if (m1 == 1)
        return a2;
    if (m2 == 1)
        return a1;
    ll t = (a2 - a1) % m2;
    if (t < 0)
        t += m2;
    t = t * inv(m1 % m2, m2) % m2;
    return (a1 + m1 * t) % (m1 * m2);
}
```

10.2 Eratosthenes

```
void erat(ll N, vector<ll>& primes, vector<ll>& mi)
    {
    mi.assign(N, 0);
    for (int i = 2; i < N; ++i) {
        if (mi[i] == 0) {
            primes.push_back(i);
            mi[i] = i;
        }

        for (int j = 0; j < primes.size() && primes[
    j] <= mi[i] && i * primes[j] < N; ++j) {
            mi[primes[j] * i] = primes[j];
        }
    }
}
```

10.3 FloorSum

```
ll floor_div(ll a, ll b) { // b > 0
    if (a >= 0)
        return a / b;
    return (a - b + 1) / b;
}

// sum_{x=0}^{n-1} floor((a*x + b) / m), m > 0
ll floor_sum(ll n, ll m, ll a, ll b) {
    if (n <= 0) return 0;
    ll ans = 0;
    auto pull = [&](ll &x, ll w) {
        ll q = floor_div(x, m);
        ans += q * w;
        x -= q * m;
    };
    while (true) {
        pull(a, n * (n - 1) / 2);
        pull(b, n);
        ll y = a * n + b;
        if (y < m) return ans;
        n = y / m;
        b = y % m;
        swap(a, m);
    }
}
```

10.4 Miller Rabin

```
bool _miller_rabin_test(ll n, ll d, ll s, ll a) {
    ll x = bin_pow_mod(a, d, n);
    for (int i = 0; i < s; ++i) {
        ll y = ((__int128_t)x * x) % n;
        if (y == 1 && x != 1) {
            return false;
        }

        x = y;
    }
    if (x != 1) {
        return false;
    }
    return true;
}

bool is_prime_ml(ll n) {
    if (n <= 1) {
        return false;
    }
    // all primes
    vector<ll> as = {2, 3, 5, 7, 11, 13, 17, 19, 23,
     29, 31, 37};
    ll d = n - 1;
    ll s = 0;
    while ((d & 1) == 0) {
```

```
        ++s;
        d >>= 1;
    }
    for (ll a : as) {
        if (n == a) {
            return true;
        }
        if (n % a == 0) {
            return false;
        }
        if (!_miller_rabin_test(n, d, s, a)) {
            return false;
        }
    }
    return true;
}
```

11 Strings

11.1 LCP

```
// p - suffix massive output
// c - suffix massive classes
vector<ll> calc_lcp(ll n, const string& s, const
    vector<ll>& c, const vector<ll>& p) {
    vector<ll> lcp(n - 1, 0);

    ll pref = 0;

    for (int i = 0; i < n - 1; ++i) {
        ll pos = c[i];
        ll j = p[pos - 1];
        while (i + pref < n && j + pref < n && s[i +
     pref] == s[j + pref]) {
            ++pref;
        }
        lcp[pos - 1] = pref;
        pref = max(0ll, pref - 1);
    }

    return lcp;
}
```

11.2 Prefix Automate

```
struct PrefixAutomate {
    const static int A = 26;

    struct Node {
        int next[A] = {-1};
        int link = -1;

        int prev = -1;
        int pe = -1;

        int has = 0;
    };

    vector<Node> tree;

    PrefixAutomate() {
        tree.push_back(Node());
    }

    void add(const string& s) {
        int v = 0;
        for (char c : s) {
            int e = c - 'a';
            if (tree[v].next[e] == -1) {
                tree[v].next[e] = tree.size();
                tree.push_back(Node());
                tree.back().prev = v;
                tree.back().pe = e;
            }
            v = tree[v].next[e];
        }
        tree[v].has = 1;
    }

    void addVect(const vector<string>& vs) {
        for (const auto& s : vs) {
            add(s);
        }
    }

    void del(const string& s) {
        int v = 0;
        for (char c : s) {
            int e = c - 'a';
            if (tree[v].next[e] == -1) {
                return;
            }
            v = tree[v].next[e];
        }
        tree[v].has = 0;
    }

    void ahoCorasick() {
        queue<int> que;

        tree[0].prev = 0;
        tree[0].pe = -1;

        tree[0].link = 0;
        for (int i = 0; i < A; ++i) {
            if (tree[0].next[i] == -1) {
                tree[0].next[i] = 0;
            } else {
                que.push(tree[0].next[i]);
            }
        }

        while (!que.empty()) {
            int v = que.front();
            que.pop();

            if (tree[v].link == -1) {
                if (v == 0 || tree[v].prev == 0)
                    tree[v].link = 0;
                else
                    tree[v].link = tree[tree[v].prev
].next[tree[v].pe];
            }

            for (int i = 0; i < A; ++i) {
                if (tree[v].next[i] == -1) {
                    tree[v].next[i] = tree[tree[v].
link].next[i];
                } else {
                    que.push(tree[v].next[i]);
                }
            }
        }
    }
};
```

11.3 Prefix Function

```
vector<ll> prefix_function(string s) {
    ll n = s.size();
    vector<ll> p(n, 0);
    for (int i = 1; i < n; ++i) {
        ll k = p[i - 1];
```

```
        while (k > 0 && s[i] != s[k])
            k = p[k - 1];
        if (s[i] == s[k])
            ++k;
        p[i] = k;
    }
    return p;
}
```

11.4 Suffix Massive

```
void radix_sort(vector<pair<pair<ll, ll>, ll>>& prs)
     {
    ll n = prs.size();
    vector<ll> cnt(n, 0);
    for (auto mem : prs) {
        cnt[mem.first.first]++;
    }
    vector<ll> ptr(n, 0);
    for (int i = 1; i < n; ++i) {
        ptr[i] = ptr[i - 1] + cnt[i - 1];
    }

    vector<pair<pair<ll, ll>, ll>> new_prs(n);
    for (auto mem : prs) {
        ll i = mem.first.first;
        new_prs[ptr[i]++] = mem;
    }
    swap(prs, new_prs);
}

// s already includes last symbol '$' (-inf)
pair<vector<ll>, vector<ll>> suffix_array(const
    vector<ll>& s) {
    ll n = s.size();
    vector<ll> p(n), c(n);

    {  // k == 0
        vector<pair<ll, ll>> prs(n);
        for (int i = 0; i < n; ++i) {
            prs[i] = {s[i], i};
        }
        sort(all(prs));
        for (int i = 0; i < n; ++i) {
            p[i] = prs[i].second;
        }

        c[p[0]] = 0;
        for (int i = 1; i < n; ++i) {
            if (prs[i].first == prs[i - 1].first) {
                c[p[i]] = c[p[i - 1]];
            } else {
                c[p[i]] = c[p[i - 1]] + 1;
            }
        }
    }

    ll k = 0;

    while ((1ll << k) < n) {
        vector<pair<pair<ll, ll>, ll>> prs(n);
        for (int i = 0; i < n; ++i) {
            ll t = p[i];
            ll j = ((t - (1ll << k)) % n + n) % n;
            prs[i] = {{c[j], c[t]}, j};
        }

        radix_sort(prs);

        for (int i = 0; i < n; ++i) {
            p[i] = prs[i].second;
        }

        c[p[0]] = 0;
        for (int i = 1; i < n; ++i) {
            if (prs[i].first == prs[i - 1].first) {
                c[p[i]] = c[p[i - 1]];
            } else {
                c[p[i]] = c[p[i - 1]] + 1;
            }
        }

        ++k;
    }

    return {p, c};
}
```

11.5 Suffix Tree

```
struct SuffixTree {
    struct Node {
        ll l = 0, r = 0, par = -1, link = -1;
        map<char, ll> to;

        Node(ll L = 0, ll R = 0, ll Par = -1) : l(L)
, r(R), par(Par), link(-1) {
        }

        ll len() const {
            return r - l;
        }
    };

    string s;
    vector<Node> nodes;
    ll n = 0, sz = 1;
    ll v = 0, pos = 0;

    ll go(ll v, ll& pos, ll l, ll r) {
        while (l < r) {
            if (pos == nodes[v].len()) {
                auto it = nodes[v].to.find(s[l]);
                if (it == nodes[v].to.end())
                    return -1;
                v = it->second;
                pos = 0;
                continue;
            }
            if (s[nodes[v].l + pos] != s[l])
                return -1;
            ll rem = nodes[v].len() - pos;
            if (r - l < rem) {
                pos += r - l;
                return v;
            }
            l += rem;
            pos = nodes[v].len();
        }
        return v;
    }

    ll split(ll v, ll pos) {
        if (pos == nodes[v].len())
            return v;
        if (pos == 0)
            return nodes[v].par;
        Node u = nodes[v];
        ll mid = sz++;
        nodes[mid] = Node(u.l, u.l + pos, u.par);
        nodes[u.par].to[s[u.l]] = mid;
        nodes[mid].to[s[u.l + pos]] = v;
        nodes[v].par = mid;
        nodes[v].l += pos;
```

```
    return mid;
}

ll get_link(ll v) {
    if (nodes[v].link != -1)
        return nodes[v].link;
    if (nodes[v].par == -1)
        return 0;
    ll to = get_link(nodes[v].par);
    ll p = nodes[to].len();
    ll nv = go(to, p, nodes[v].l + (nodes[v].par
 == 0), nodes[v].r);
    return nodes[v].link = split(nv, p);
}

void extend(ll i) {
    while (true) {
        ll p = pos;
        ll nv = go(v, p, i, i + 1);
        if (nv != -1) {
            v = nv;
            pos = p;
            return;
        }
        ll mid = split(v, pos);
        ll leaf = sz++;
        nodes[leaf] = Node(i, n, mid);
        nodes[mid].to[s[i]] = leaf;
        v = get_link(mid);
        pos = nodes[v].len();
        if (mid == 0)
            break;
    }
}

SuffixTree() {
}

SuffixTree(string str) {
    s = str;
    n = s.size();
    nodes.assign(2 * n + 5, Node());
    nodes[0] = Node(0, 0, -1);
    sz = 1;
    v = 0;
    pos = 0;
    for (ll i = 0; i < n; ++i)
        extend(i);
    }
};
```

11.6 Z Function

```
vector<ll> z_function(string s) {
    ll n = s.size();
    vector<ll> z(n, 0);
    ll r = 0;
    for (int i = 1; i < n; ++i) {
        if (z[i - r] < r + z[r] - i)
            z[i] = z[i - r];
        else {
            z[i] = max(r + z[r] - i, 0ll);
            while (i + z[i] < n && s[i + z[i]] == s[
    z[i]])
                ++z[i];
            r = i;
        }
    }
    return z;
}
```